CSE 4401: Software Development and Management

Contact Hours: 45 hours
Prerequisites: CSE 4203 Advanced Object – Oriented Programming, CIT 4302 Advanced Web Programming
Purpose: This course gives students experience designing, implementing, testing, and debugging large programs

Expected Learning Outcomes
At the end of this course the student should be able to:

i. Know software process models and software project management
ii. Understand software development phases: requirements and analysis, design and construction, testing, deployment, operations, and maintenance
iii. Understand modern software development and management platforms, tools, and services

Course Content
Introduction to Software Design paradigms, Project Management, System and Software Requirements, Computer Aided Software Engineering, and Software Design Fundamentals using existing documentation for a proposed system. In-depth survey of data flow-oriented, object-oriented, data oriented, and real-time design. Team project involving the implementation of the proposed system using structured programming, information hiding, and strength and coupling measures. Software reviews. Software testing techniques and strategies. Software maintenance. Each student will be required to make an oral presentation as part of the team project.

Mode of Delivery
Lectures, tutorials.

Instructional Materials /Equipment
A computer laboratory, lecture notes, illustration charts, journals, overhead presentation equipment.
Assessment

Type	Weighting (%)
Examination	70%
Continuous Assessment	30%
Total	100%

Core Text Books

i. Sommerville, I. (2016). *Software engineering* (10th ed.). Boston, MA: Pearson Education. ISBN: 1292096136.
ii. Pressman, R. (2014). *Software engineering* (8th ed.). New York: Mcgraw-Hill. ISBN: 1259175758.
iii. Braude, E. J. (2016). *Software Engineering: Modern Approaches* (2nd ed.). Long Grove: Waveland Press, Inc. ISBN: 1478632305.

Core Journals

i. *International Journal of Software Engineering and Knowledge Engineering. ISSN: 0218-1940.*
ii. *ACM Transactions on Software Engineering and Methodology. ISSN: 1049-331X.*
iii. *Advances in Engineering Software. ISSN: 0965-9978.*

Recommended Textbooks

i. Stephens, R. (2015). *Beginning software engineering* (1st ed.). Hoboken, NJ: John Wiley & Sons. ISBN: 1118969146.

ii. Klimczak, E. (2013). *Design for Software: A Playbook for Developers* (1st ed.). West Sussex, UK: John Wiley & Sons. ISBN: 111994290X.

iii. Wiegers, K. E., & Beatty, J. (2015). *Software requirements* (3rd ed.). Redmond, WA: Microsoft Press. ISBN: 0735679665.

Recommended Journals

i. *Journal of Systems and Software. ISSN: 0164-1212.*

ii. *IEEE Software. ISSN: 0740-7459.*

iii. *International Journal of Software Engineering, Technology and Applications. ISSN: 2053-2466.*

COURSE TITLE: **SOFTWARE DEVELOPMENT AND MANAGEMENT**
PAPER CODE: **CSE 4401**
LECTURER: **MASESE CHUMA**
CONTACT: **0782526000**
TOPIC: **INTRODUCTION**

Software is more than just a program code. A program is an executable code, which serves some computational purpose. Software is considered to be collection of executable programming code, associated libraries and documentations. Software, when made for a specific requirement is called **software product.**

Engineering on the other hand, is all about developing products, using well-defined, scientific principles and methods

Software engineering is an engineering branch associated with development of software product using well-defined scientific principles, methods and procedures.

The outcome of software engineering is an efficient and reliable software product.

Software:

a) Instructions (Computer Programs) that when executed provide desired features, function, and performance.

b) Data structures that enable the programs to adequately manipulate information.

c) Documents that describe the operation and use of the programs.

IEEE defines software engineering as:

(1) The application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.

(2) The study of approaches as in the above statement. Fritz Bauer, a German computer scientist, defines software engineering as:

"Software engineering is the establishment and use of sound engineering principles in order to obtain economically software that is reliable and work efficiently on real machines."

Software Engineering

According to Firtz Bauer – Software engineering is defined as "*The establishment and use of sound engineering principles in order to obtain the software that is economical, reliable and work efficiently on real machines".*

According to Boehm – "Software engineering is the application of science and mathematics by which capabilities of computer equipment are made useful to man via computer programs, procedures and related documentation".

According to IEEE – Software engineering is the systematic approach to the development, operation, maintenance and retirement of the software.

"*Software engineering is the technological and managerial discipline concerned with systematic production and maintenance of software products that are developed and modified on time and within cost estimates*".

Evolutionary Role of Software

Software performs 2 roles

1. Product – As a product, it delivers the computing potential embodied by computer hardware or, more broadly, a network of computers that are accessible by local hardware.
2. Vehicle/ Platform – As a vehicle used to deliver the product, software acts as the basis for the control of the computer (operating systems), the communication of information (networks), and the creation and control of other programs (software tools and environments).

Functions provided by Software

1. It delivers most important product i.e. Information or Data.
2. It provides the Gateway for worldwide information.
3. It provides the means of exchanging information.
4. It manages the business Information.

Software Applications (Categories of Computer Software)

Software applications run on different areas; they are as follows

1. **System Software**

System software is a collection of programs written to service other programs.

E.g. Compilers, Editors, File Management Utilities.

2. **Real-time software**

Software that monitors/ analyzes/ controls real-world events as they occur is called real time Software.

E.g. Real time manufacturing process control.

3. **Business Software**

Business information processing is the largest single software application area. They include software that accesses one or more large databases containing business information.

E.g. Payroll, Inventory, Accounts.

4. **Engineering and scientific software**

Engineering & Scientific software is the software required for the engineering & scientific development purpose. It will include all CASE tools & System Stimulations.

E.g., CAD, CAM.

5. **Embedded Software**

Embedded software resides in read-only memory and is used to control products and systems for the consumer and industrial markets.

E.g. digital functions in an automobile such as fuel control, dashboard displays, and braking systems

6. **Personal Computer Software**

These soft wares are used for enhancing the personal computers & to facilitate more control to the user.

E.g. Computer Graphics, Multimedia, Text Editors.

7. **Web Based Software**

The Web pages retrieved by a browser are software that incorporates executable instructions (e.g., CGI, HTML, Perl, or Java), and data.

8. **Artificial Intelligence Software**

Software makes use of non-numerical algorithms to solve complex problems that are not amenable to computation or straightforward analysis.

E.g. Expert Systems, Pattern Recognition.

SOFTWARE ENGINEER

A software engineers is a person who applies a broad range of application development knowledge to the systematic development of application systems for organizations. Software engineers used to think of their job as conscientious development of well-structured computer programs. But, as the field evolved, systems analysis as a task appeared along with systems analysts, the people who perform that task. Now, ***there is a proliferation of techniques, tools, and technologies to develop applications.*** Software engineers' jobs have evolved to now include evaluation, selection, and use of specific systematic approaches to the development, operation, maintenance, and retirement of software.

Fundamental skills of software engineers include

1. How to identify, evaluate, choose, and implement an appropriate methodology And CASE tools
2. How and when to use prototyping
3. How and when to select hardware, software, and languages
4. How to manage activities associated with configuration management, planning, and control of the development process

5. How to select computer languages and develop computer programs
6. How and which project testing techniques to apply
7. How to choose and use software maintenance techniques
8. How to evaluate and decide when to retire Applications

The goals of a software engineer are to produce a high-quality product and to enjoy a high-quality development process. The *product* of a software engineering effort is a delivered, working computer
system, some examples of which include:

- Accounts receivable processing
- Order processing
- Inventory monitoring and maintenance
- Decision support for overnight funds investment
- Collateralized mortgage obligation cost determination
- Insurance reimbursement processing
- Funds transfer processing
- Early warning system for problems with critical success factors
- Query processing for a customer information database

A quality SE product is

- on time
- within budget
- functional, i.e., does what it is supposed to do
- friendly to users
- error free
- flexible
- adaptable

<u>Goals of Software Engineering</u>

The primary goals of Software Engineering is:

1. To improve the quality of software.
2. To increase productivity.
3. To increase the job satisfaction of software engineers.

Software engineering is based on the following disciplines

1. Computer Science.
2. Management Science.
3. Economics.
4. Communication Skills.
5. Engineering approach to problem solving.

Software Engineering is a labor-intensive activity where high degree of communication is required among

1. Customers.
2. Managers.
3. Software Engineers.
4. Hardware Engineers.
5. Other related technologists.

Need of Software Engineering

The need of software engineering arises because of higher rate of change in use requirements and environment on which the software is working. Following are some of the needs stated:

- **Large software -** It is easier to build a wall than a house or building, likewise, as the size of the software becomes large, engineering has to step to give it a scientific process.
- **Scalability-** If the software process were not based on scientific and engineering concepts, it would be easier to re-create new software than to scale an existing one.
- **Cost-** As hardware industry has shown its skills and huge manufacturing has lower down the price of computer and electronic hardware. But, cost of the software remains high if proper process is not adapted.
- **Dynamic Nature-** Always growing and adapting nature of the software hugely depends upon the environment in which the user works. If the nature of software is

always changing, new enhancements need to be done in the existing one. This is where the software engineering plays a good role.

- **Quality Management-** Better process of software development provides better and quality software product.

Characteristics of good software

A software product can be judged by what it offers and how well it can be used.
This software must satisfy on the following grounds:

- Operational
- Transitional
- Maintenance

Well-engineered and crafted software is expected to have the following characteristics:

Operational

This tells us how well the software works in operations. It can be measured on:

- Budget
- Usability
- Efficiency
- Correctness
- Functionality
- Dependability
- Security
- Safety

Transitional

This aspect is important when the software is moved from one platform to another:

- Portability
- Interoperability
- Reusability

- Adaptability

Maintenance

This aspect briefs about how well the software has the capabilities to maintain itself in the ever-changing environment:

- Modularity
- Maintainability
- Flexibility
- Scalability

Software engineering is a branch of computer science, which uses well defined engineering concepts required to produce efficient, durable, scalable, in budget, and on-time software products.

SOFTWARE EVOLUTION

The process of developing a software product using software engineering principles and methods is referred to as **Software Evolution.** This includes the initial development of software and its maintenance and updates, till desired software product is developed, which satisfies the expected requirements.

Evolution starts from the requirement gathering process. After which developers create a prototype of the intended software and show it to the users to get their feedback at the early stage of the software product development. The users suggest changes, on which

several consecutive updates and maintenance keep on changing too. This process changes to the original software, till the desired software is accomplished.

Even after the user has the desired software in hand, the advancing technology and the changing requirements force the software product to change accordingly.

Re-creating software from scratch and to go one-on-one with the requirement is not feasible. The only feasible and economical solution is to update the existing software so that it matches the latest requirements.

Software Evolution Laws

Lehman has given laws for software evolution. He divided the software into three different categories:

1. **Static-type (S-type) -** This is a software, which works strictly according to defined specifications and solutions. The solution and the method to achieve it, both are immediately understood before coding. The s-type software is least subjected to changes hence this is the simplest of all. For example, calculator program for mathematical computation.
2. **Practical-type (P-type) -** This is a software with a collection of procedures. This is defined by exactly what procedures can do. In this software, the specifications can be described but the solution is not obviously instant. For example, gaming software.
3. **Embedded-type (E-type) -** This software works closely as the requirement of real-world environment. This software has a high degree of evolution as there are various changes in laws, taxes etc. in the real-world situations. For example, Online trading software.

E-Type software evolution

Lehman has given eight laws for E-Type software evolution -

1. **Continuing change -** An E-type software system must continue to adapt to the real-world changes, else it becomes progressively less useful.
2. **Increasing complexity -** As an E-type software system evolves, its complexity tends to increase unless work is done to maintain or reduce it.
3. **Conservation of familiarity -** The familiarity with the software or the knowledge about how it was developed, why was it developed in that particular manner etc., must be retained at any cost, to implement the changes in the system.

4. **Continuing growth-** In order for an E-type system intended to resolve some business problem, its size of implementing the changes grows according to the lifestyle changes of the business.

5. **Reducing quality -** An E-type software system declines in quality unless rigorously maintained and adapted to a changing operational environment.

6. **Feedback systems-** The E-type software systems constitute multi-loop, multi-level feedback systems and must be treated as such to be successfully modified or improved.

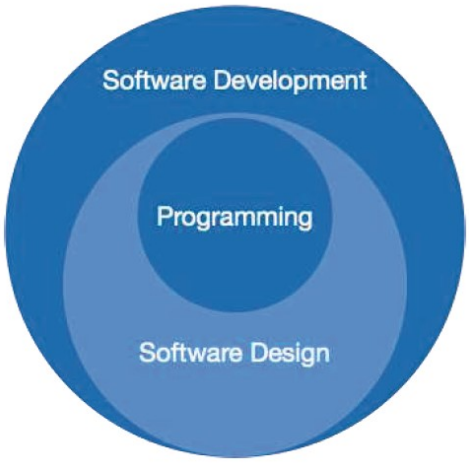
7. **Self-regulation -** E-type system evolution processes are self-regulating with the distribution of product and process measures close to normal.

8. **Organizational stability -** The average effective global activity rate in an evolving E-type system is invariant over the lifetime of the product.

Software Paradigms

Software paradigms refer to the methods and steps, which are taken while designing the software. There are many methods proposed and are implemented.

But, we need to see where in the software engineering concept, these paradigms stand. These can be combined into various categories, though each of them is contained in one another:



Programming paradigm is a subset of Software design paradigm which is further a subset of Software development paradigm.

Software Development Paradigm

This paradigm is known as software engineering paradigms; where all the engineering concepts pertaining to the development of software are applied. It includes various

researches and requirement gathering which helps the software product to build. It consists of –

- Requirement gathering
- Software design
- Programming

Software Design Paradigm

This paradigm is a part of Software Development and includes –

- Design
- Maintenance
- Programming

Programming Paradigm

This paradigm is related closely to programming aspect of software development.
This includes –

- Coding
- Testing
- Integration